Lecture 26.12: KNN Classifier Implementation from Scratch

📌 Topic Overview

In this lecture, Shraddha ma'am writes the complete production-grade Python class for **k-Nearest Neighbors from Scratch**, verifying its accuracy against synthetic benchmark data.

🏛️ Class Architecture: KNNClassifierScratch

- `__init__(self, k=3)` : Initializes the neighbor count hyperparameter.
- `fit(self, X, y)` : Caches the training matrices in memory.
- `_predict_single(self, x)` : Computes vector distances $\sqrt{\sum(X-x)^2}$, grabs top k labels using `np.argsort`, and determines the majority class with `np.bincount`.
- `predict(self, X)` : Loops over incoming test rows and returns final predictions.

💻 Python Implementation

```
import numpy as np

class KNNClassifierScratch:
    """
    Complete Vectorized Scratch Implementation of k-Nearest Neighbors:
    - fit(X, y): Stores training data
    - predict(X): Vectorized distance computation, sorting, and majority voting
    """
    def __init__(self, k=3):
        self.k = k
        self.X_train = None
        self.y_train = None

    def fit(self, X, y):
        self.X_train = np.asarray(X)
        self.y_train = np.asarray(y)
        return self

    def _predict_single(self, x):
        # 1. Compute Euclidean distances to all training points
        distances = np.sqrt(np.sum((self.X_train - x) ** 2, axis=1))
```

```
        # 2. Get indices of k nearest neighbors
        k_indices = np.argsort(distances)[:self.k]
        k_nearest_labels = self.y_train[k_indices]

        # 3. Majority vote
        counts = np.bincount(k_nearest_labels)
        return np.argmax(counts)

    def predict(self, X):
        X = np.asarray(X)
        predictions = [self._predict_single(x) for x in X]
        return np.array(predictions)

# --- Function Call & Testing ---
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_raw, y_raw = make_classification(n_samples=300, n_features=4, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X_raw, y_raw, test_size=0.2, r

# Scale features
scaler = StandardScaler()
X_tr_s = scaler.fit_transform(X_train)
X_te_s = scaler.transform(X_test)

knn = KNNClassifierScratch(k=5)
knn.fit(X_tr_s, y_train)
preds = knn.predict(X_te_s)
acc = np.mean(preds == y_test)

print(f"Scratch KNN Classifier (k=5) Accuracy: {acc*100:.2f}%")
```

🔑 Key Takeaways

1. Scratch kNN perfectly replicates Scikit-Learn's `KNeighborsClassifier` behavior.
2. Even in scratch implementations, feature scaling with `StandardScaler` remains mandatory.